Dgrammar: Efficient Constrained Decoding for DLMs

Anonymous ACL submission

Abstract

Grammar-constrained decoding for DLMs remains slow because existing decoders enforce constraints one token at a time, turning violations into rejection, lookahead verification, or autoregressive fallback. This discards the block-parallel decoding advantage of DLMs. We introduce Dgrammar, which preserves multi-token unmasking and repairs violations in place under the same forward-pass logits. Dgrammar combines incremental prefix checking, selective remasking via logit truncation, asynchronous mask construction, and grammar-guided tail completion. We also characterize how rejection filtering, lookahead verification, and AR fallback change the model's scoring distribution. On JSONSchemaBench medium_test with LLaDA-8B-Instruct, Dgrammar improves schema-validity from 76.3% to 84.5%, reduces mean latency by 5.8×, reduces p95 latency by 9.4×, and eliminates 120 s timeouts.

1 Introduction

Masked diffusion language models such as LLaDA (Nie et al., 2025) and Dream (Ye et al., 2025) reach the quality of autoregressive (AR) models of comparable size while decoding multiple tokens per step. This parallel-decoding property is their main systems advantage: in a single forward pass the model produces a joint distribution over all masked positions, and any subset of high-confidence positions can be unmasked at once.

Many downstream uses of LLMs require structured output that obeys a formal grammar, for example JSON conforming to a schema, code, or domain-specific notations (Willard and Louf, 2023; Dong et al., 2024). AR constrained decoding is well studied: a token-level matcher such as XGrammar reports per-token mask-construction cost on the order of microseconds, well below a forward pass.

For DLMs the picture is different. Existing methods such as ETH/IG-CD (Mündler et al., 2025) and LAVE (Zhang et al., 2026) enforce grammar by reducing the number of tokens unmasked per step to one. Violations are handled as rejection or verification events: ETH/IG-CD suppresses rejected candidates and resamples, while LAVE uses beam-search lookahead and can trigger an AR fallback after repeated rejection. This reduces the parallel-decoding budget in two ways. First, the per-step throughput drops by the maximum batch factor. Second, each violation triggers extra verification, and the AR fallback runs the diffusion model in a regime never seen during training. The headline numbers are reported only as total wall time, which obscures both the per-token constraint cost and the heavy tail caused by fallbacks.

We make two contributions: (i) we argue that per-token constraint overhead is a useful axis for comparing grammar-constrained DLM methods, in line with how AR work like XGrammar (Dong et al., 2024) and llguidance (Microsoft, 2024) is reported, and give per-operation timing breakdowns for grammar checks, mask construction, and their ratio to the forward pass; and (ii) we introduce Dgrammar, whose core idea is *single-frontier repair*: a grammar violation is fixed in place at the first offending token under the same forward-pass logits, rather than rejecting the whole parallel step, so multi-token unmasking is preserved. On top of this principle we add two semantics-preserving systems accelerations, asynchronous overlap of mask construction with the forward pass and a bounded grammar-guided AR tail, and isolate the principle's gains from the accelerations' in Table 2.

On JSONSchemaBench medium_test (Geng et al., 2025) with LLaDA-8B-Instruct on a B200 GPU, Dgrammar improves schema-valid rate from 76.3% (LAVE) to 84.5% while reducing mean latency from 30.6 s to 5.2 s and p95 latency from 120 s to 12.8 s. The fraction of instances that hit the

120 s timeout drops from 12.7% to 0%.

2 Background and Related Work

Masked DLMs. A masked diffusion model predicts every position from a partially masked sequence. At inference, decoding starts from an all-mask sequence of length L and runs for T steps; at each step the model produces logits for all L positions in one forward pass and the decoder unmasks K positions chosen by a confidence rule. With $K > 1$ this is strictly faster per step than AR generation. DLMs such as LLaDA (Nie et al., 2025) and Dream (Ye et al., 2025) reach AR-comparable quality this way; parallel-decoding accelerators (Wu et al., 2025; Chen et al., 2026, 2025) adapt K per step but remain unconstrained; Dgrammar's mask-based repair is in principle compatible with such schedules.

Grammar-constrained AR decoding. Token-level matchers (Willard and Louf, 2023; Dong et al., 2024; Microsoft, 2024; Geng et al., 2023; Park et al., 2025) maintain an incremental parser state and return a vocabulary mask of allowed next tokens at every step. Per-token mask construction is in the microsecond range, well below the cost of a forward pass. Park et al. (2024) further analyze how greedy mask truncation alters the model's scoring distribution under grammar constraints, motivating our framing in §3. These interfaces do not transfer to diffusion because there is no single causal frontier until we define one.

Grammar-constrained DLM decoding. ETH/IG-CD (Mündler et al., 2025) suppresses rejected candidates by checking whether the partial output still admits a grammar-valid completion. LAVE (Zhang et al., 2026) replaces this verifier with a beam-search lookahead and falls back to per-position AR decoding after repeated rejection; it reports 99% syntactic validity on `json-mode-eval`, but on harder splits validity drops sharply (§5). DINGO (Suresh et al., 2025) runs full-sequence dynamic programming at higher per-step cost, while plan- and draft-style decoders (Li et al., 2026; Reddy et al., 2026) emphasize structure-aware planning. Both ETH/IG-CD and LAVE unmask one token per step and report only end-to-end runtime.

3 Where Each Decoder Intervenes

We map where each constrained DLM decoder applies its grammar bias relative to the model's raw forward-pass logits. This taxonomy is descriptive: it clarifies *what is being changed* before the empirical comparison in §5.

Let $\ell \in \mathbb{R}^{L \times V}$ be the logits from one forward pass on the current block context $\mathbf{x}$. Write

$$P_p(t) \;=\; p_\theta(t \mid \mathbf{x}, p) \;=\; \mathrm{softmax}(\ell[p, \cdot])_t$$

for the model's per-position distribution at masked position p. Let $G_p \subseteq V$ be the token set accepted by the incremental matcher at the current prefix frontier, and define the grammar-truncated block-context distribution

$$P_p^G(t) \;=\; \frac{\mathbf{1}[t \in G_p] P_p(t)}{\sum_{u \in G_p} P_p(u)} \quad \text{when } P_p(G_p) > 0.$$

Any grammar-constrained decoder must replace P_p by some grammar-biased scoring rule. The distinction is whether this bias is applied to the same block-context logits or after changing the scoring objective. This section is a qualitative map of *where* each decoder injects its bias, not a quantitative theory of quality: the total-variation terms below name the relevant distances, but their consequences for validity and latency are settled empirically (§5), and Proposition 1 is a local correctness statement, not a claim about the constrained joint distribution.

Proposition 1 (Local MAP under truncation)**.** *Assume $G_p \neq \emptyset$, the matcher state is restored between retries, and the retry cap is not reached. Then the logit-truncation loop selects, in at most $|V \setminus G_p|$ rejected candidates,*

$$\hat{t}_p^D \;=\; \arg\max_{t \in G_p} \ell[p, t] \;=\; \arg\max_{t \in G_p} p_\theta(t \mid \mathbf{x}, p).$$

Proof. Each retry picks $\arg\max \ell[p]$ under the current suppression set. If the token is in G_p, the loop exits; otherwise its logit is set to $-\infty$. After all higher-scoring tokens outside G_p have been suppressed, the best remaining token is the constrained MAP under the original forward-pass logits. □

Proposition 1 characterizes one repair subroutine: *which* token Dgrammar selects after a single-position violation. It is not a global guarantee about distribution preservation or downstream quality.

Where the baselines apply their bias. The other DLM-constrained decoders apply their grammar bias to a different object. ETH/IG-CD performs rejection filtering with a completion test: its accepted token is $\hat{t}_p^{\mathrm{ETH}} = \arg\max_{t \in C_p(\mathbf{x})} P_p(t)$, the MAP over the feasible set $C_p(\mathbf{x}) = \{t \in V : \mathrm{Comp}(\mathbf{x}, p, t, \mathcal{G}) = 1\}$ of tokens whose insertion leaves a nonempty grammar intersection. This still uses the same forward-pass logits, but the feasibility set depends on the whole partially filled block and the sequential commitment order, not only the current frontier. The induced change is $\mathrm{TV}(P_p^G, P_p^C)$ (P_p^C normalizes P_p over C_p); this may be zero, and we claim no universal ordering.

LAVE changes the objective more directly: its verifier selects $\hat{t}_p^{\mathrm{LAVE}} = \arg\max_t s_{\mathrm{LAVE}}(t; \mathbf{x}, \mathcal{G})$ through a beam-lookahead score over future masked positions, which need not coincide with $\arg\max_t P_p(t)$ or $\arg\max_{t \in G_p} P_p(t)$. If it falls back to per-position AR decoding, the conditioning changes to $Q_p(t) = p_\theta(t \mid \mathbf{x}_{<p})$ (grammar-truncated to Q_p^G), so the relevant distortion is the context-switch mismatch $\mathrm{TV}(P_p^G, Q_p^G)$, with $\mathrm{TV}(p, q) = \frac{1}{2}\sum_t |p(t) - q(t)|$. Neither term implies token-level error: different distributions can share the same MAP token.

In summary, Dgrammar applies grammar bias inside the same block-context logits; ETH/IG-CD filters candidates through a completion set; LAVE adds a beam-lookahead objective; and AR fallback changes the conditioning context. Which design yields higher validity, cheaper interventions, and fewer failures is tested empirically in §5.

4 Method

Dgrammar runs the standard masked-diffusion loop but changes how grammar feedback is used. Instead of treating a violation as a rejection of the whole step, it preserves accepted tokens, repairs the first invalid frontier token under the same forward-pass logits, and uses parser feedback to control how aggressively the next step unmasks tokens. Algorithm 1 gives the full step; Figure 1 contrasts it with LAVE.

4.1 Prefix-Guided Selective Remasking

The decoder maintains a left-to-right cursor c into the generated region and a token-level matcher state (Microsoft, 2024). After each batch unmask, ExtendPrefix feeds the contiguous non-mask suffix starting at c to the matcher. If all tokens are consumed, c advances. Otherwise, the first unconsumed position p is the frontier violator; all earlier positions have already been accepted by the matcher.

The repair is local. Let t_p be the rejected token at p. Dgrammar sets $\ell[p, t_p] = -\infty$, returns x_p to `[MASK]`, takes the next argmax at p, and retries the matcher from the same accepted prefix. This loop uses the same forward-pass logits and costs only a vector argmax plus one matcher call per retry. It stops when the matcher accepts a replacement or when `max_resamples` is reached. Tokens already accepted in the same batch are not re-decoded, so a single violation becomes a refinement target rather than a rollback to single-token decoding.

4.2 Grammar-Gated Parallel Unmasking

The batch size K is a parser-controlled state variable updated by an additive-increase, multiplicative-decrease (AIMD) rule familiar from TCP congestion control: after a violation-free step we set $K \leftarrow \min(2K, K_{\max})$; after a violation we reset K to one. This is analogous to confidence-adaptive schedules for diffusion decoding (Wu et al., 2025), but the signal is grammatical progress rather than softmax confidence. Easy spans therefore recover parallel unmasking quickly, while schema-sensitive regions such as delimiters, field names, and type-restricted values are decoded conservatively. The AIMD schedule, the asynchronous overlap of ComputeMask with the GPU forward pass, and the per-backbone adapter are detailed in App. E.

Grammar constraints are applied only at the current frontier. Before token selection, ComputeMask returns the vocabulary bias allowed by the matcher state at cursor c, and this bias is added to $\ell[c]$. Non-frontier positions are selected by the usual diffusion confidence rule. This frontier-only masking avoids constructing masks for every masked position in the block while still ensuring that the next token consumed by the parser is grammar-valid or becomes a selective-remasking target.

4.3 Tail and Latency Control

Two engineering choices keep constraint overhead from dominating the diffusion loop. First, mask construction for the current frontier is independent of the next model forward pass, so we launch ComputeMask on a background thread and join it after the forward pass. We fall back to synchronous

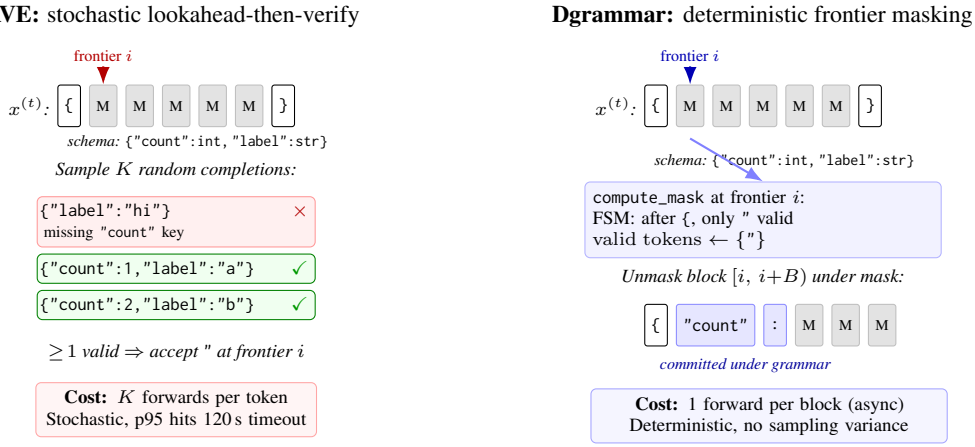

Figure 1: One denoising step under schema `{"count":int, "label":str}`. **LAVE** (Zhang et al., 2026) (left) samples K random completions at each frontier token, accepting it only when ≥ 1 suffix is valid; stochastic and $K\times$ more expensive per token, hitting the 120 s per-instance timeout on hard schemas. **Dgrammar** (right) runs `compute_mask` on the incremental FSM state and unmasks an entire block under that mask in one forward pass, overlapped with the next mask computation; the result is deterministic. The figure is illustrative and depicts LAVE's enum/format failure regime (App. C), where a random completion rarely hits a required token; on easier fields LAVE often commits in one or a few draws. Its actual cost depends on the retry schedule and verifier policy (§2).

mask computation only when a mid-step repair changes the matcher state. In our runs, this hides almost all mask construction latency behind GPU compute.

Second, if the diffusion loop exits before the matcher reaches an accepting state, we invoke a grammar-guided AR completion once from the current cursor, restricted to the unfinished suffix and refreshing the model logits at every emitted token under the grammar mask. In practice this mostly closes short tails such as braces or brackets; it is deliberately bounded, and failures from locally valid but schema-invalid completions are counted by the final `jsonschema.validate` metric. Algorithm 1 summarizes the step-level procedure.

Algorithm 1 Dgrammar decoding step

Input: sequence x, cursor c, matcher M, batch K
Launch ComputeMask(M) asynchronously → `pending`
$\ell \leftarrow$ Forward(x) ▷ forward pass
`bias` ← join `pending`
$\ell[c] \leftarrow \ell[c] +$ `bias` ▷ frontier mask
Choose top-K confident masked positions, write argmax tokens
$c', v \leftarrow$ ExtendPrefix(M, x, c)
if $v \neq -1$ **then** ▷ violation at v
  $\ell[v, x_v] \leftarrow -\infty$; $x_v \leftarrow$ [MASK]
  while retries $<$ `max_resamples` **do**
    $t \leftarrow \arg\max \ell[v]$
    if M.TryConsume($[t]$) $= 1$ **then** $x_v \leftarrow t$; **break**
    end if
    $\ell[v, t] \leftarrow -\infty$
  end while
  $K \leftarrow 1$
else
  $c \leftarrow c'$; $K \leftarrow \min(2K, K_{\max})$
end if
if M.IsAccepting() **then** fill remaining with EOS
end if

5 Experiments

5.1 Main JSB Results

Setup. We evaluate on JSONSchemaBench (JSB) medium_test (Geng et al., 2025); after matcher compilation skips 75 schemas, 511 instances are common to all methods (App. B). All runs use an NVIDIA B200, T=128 steps, generation length 256, block length 32, temperature 0.2, and `seed=0`. We compare Dgrammar with Vanilla unconstrained generation and LAVE (Zhang et al., 2026) on LLaDA-8B-Instruct (Nie et al., 2025) and Dream-v0-Instruct-7B ("Dream-7B") (Ye et al., 2025). Dgrammar uses one per-backbone setting for `max_resamples` (500 for LLaDA, 2000 for Dream) and per-position AR autocomplete; LAVE uses its published default `max_retry_num_total`=1000.

Metric. We report *schema@1*, the success of `jsonschema.validate` on the parsed output, applied identically to every method (Vanilla, LAVE, IG-CD, Dgrammar) and matching JSONSchemaBench (Geng et al., 2025). This is stricter than the runner-internal grammar flag, which only checks the matcher's accepting state and can be inflated by truncated outputs (App. D).

Split	Method	n	sch@1	mean	p50	p90	p95	p99	Timeout
				LLaDA-8B-Instruct					
easy	Vanilla	558	64.2	2.58	2.48	3.23	3.38	4.02	0
	IG-CD†	449	79.1	2.58	2.31	3.97	5.12	9.60	0
	LAVE	558	82.3	24.95	2.74	120.00	120.00	120.00	83
	DGR.	558	**94.1**	**3.14**	**2.71**	**4.48**	**5.36**	**11.17**	**0**
medium	Vanilla	511	51.3	3.91	3.67	5.14	5.61	7.59	0
	IG-CD†	383	67.4	9.65	3.91	17.97	44.55	106.89	4
	LAVE	511	76.3	30.56	12.32	120.00	120.00	120.00	65
	DGR.	511	**84.5**	**5.24**	**4.39**	**8.56**	**12.76**	**15.54**	**0**
hard	Vanilla	269	24.9	9.30	7.52	15.33	17.69	25.45	0
	IG-CD†	245	38.8	39.97	14.82	120.00	120.00	120.19	50
	LAVE	269	53.2	45.01	22.42	120.00	120.00	120.00	56
	DGR.	269	**54.3**	**24.17**	**19.46**	**46.50**	**54.44**	**86.21**	**0**
				Dream-7B					
easy	Vanilla	552	65.0	1.20	0.98	2.26	2.45	2.62	0
	IG-CD†	449	87.3	1.70	1.32	3.41	3.93	5.81	0
	LAVE	552	83.5	31.18	10.28	120.00	120.00	120.00	65
	DGR.	552	**94.0**	**5.29**	**3.27**	**11.12**	**14.01**	**23.89**	**0**
medium	Vanilla	506	41.5	2.71	2.79	3.81	4.17	5.16	0
	IG-CD†	383	66.8	5.96	3.33	8.40	12.96	90.69	3
	LAVE	506	78.3	34.20	15.37	120.00	120.00	120.00	58
	DGR.	506	**82.2**	**9.11**	**6.21**	**17.55**	**21.18**	**44.20**	**0**
hard	Vanilla	258	12.4	5.99	5.29	9.22	10.94	14.12	0
	IG-CD†	245	37.6	33.71	8.90	123.15	131.02	150.40	35
	LAVE	258	56.2	41.01	18.89	120.00	120.00	120.00	43
	DGR.	258	**56.6**	**18.23**	**17.73**	**37.16**	**47.38**	**51.17**	**0**

Table 1: JSB difficulty spectrum, B200, T=128. DGR. denotes Dgrammar. Latencies in seconds; Timeout counts ≥ 120 s instances. n is matched-instance count after llguidance skips. †IG-CD (Mündler et al., 2025) compiles only a subset of each split (449/577 easy, 383/586 medium, 245/368 hard, i.e. 22–35% rejected, identical across backbones); its rows are over that compilable subset, so schema@1 is not instance-matched to the other methods. On Dream its soft (`stopit`) timeout cannot interrupt a running kernel, so a few hard instances overshoot the 120 s budget. Hyperparameters in App. A.

Main results. Table 1 compares the systems. On LLaDA, VANILLA reaches only 51.3% schema@1, while LAVE improves validity to 76.3% at 7.8× higher mean latency and with 65/511 timeouts. Dgrammar raises schema@1 to 84.5%, runs 5.8× faster than LAVE on mean latency and 9.4× faster on p95, and has zero timeouts. Its mean latency is only 1.3× above VANILLA, indicating that grammar enforcement adds little cost relative to diffusion forward passes. ETH/IG-CD (Mündler et al., 2025) is not instance-matched: its CFG pipeline rejects 22–35% of each split (Table 1, †), and even on the schemas it compiles it trails Dgrammar at every difficulty (79.1 vs. 94.1, 67.4 vs. 84.5, 38.8 vs. 54.3 schema@1) with 50/245 timeouts on hard; we discuss it and DINGO in Limitations.

5.2 Ablations and Failure Analysis

Table 2 ablates Dgrammar on the same 511 instances. Async overlap never changes validity (zero per-instance flips in both blocks); its latency effect depends on the fallback, because overlap can only hide mask construction behind the block-diffusion forward loop. Under the greedy block-completion fallback the tail stays in that loop, so overlap reduces mean and p95 latency by 21% and 60%. Under per-position AR autocomplete the tail is a sequence of one-forward-per-token steps that cannot be overlapped, so the async machinery only adds thread-synchronization overhead; its mean latency is therefore within noise of the no-overlap variant (5.24 vs. 4.97 s), with the overall system still dominated by the AR tail. Autocomplete is the main validity-recovery mechanism, with schema@1 dropping from 84.5% to 75.7% when it is disabled under the AR setting; it fires on only 24.1% of instances and contributes 8.8% of all emitted tokens (8,928 of 100,997), so the diffusion loop still does the bulk of the work. Multi-token unmasking mainly saves forward passes: forcing

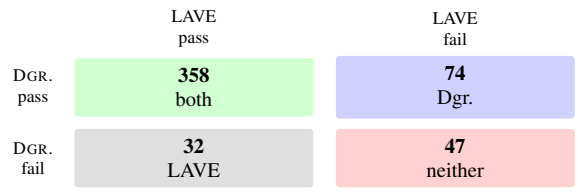

Figure 2: Paired schema@1 outcomes on JSB medium_test (LLaDA-8B-Instruct, n=511). Off-diagonal cells show instances solved by only one decoder.

$K_{\max}$=1 keeps validity nearly flat but raises mean latency by 144% under AR autocomplete. Finally, `max_resamples` is a validity/latency knob whose effect depends on the fallback: under greedy it trades validity for speed, while under AR autocomplete lower budgets shift work into slower per-token completion.

Config	schema@1	mean	p95
AR autocomplete (Table 1 setting)			
Dgrammar (full)	**84.5%**	**5.24**	**12.76**
− async overlap	84.5%	4.97	12.69
− AUTOCOMPLETE	75.7%	5.94	11.13
$K_{\max}$=1	84.0%	12.79	37.30
`max_resamples`=100	84.1%	9.55	26.82
`max_resamples`=50	83.6%	11.55	31.49
Greedy block-completion fallback			
Dgrammar (full)	81.0%	6.35	14.02
− async overlap	81.0%	7.71	22.38
− AUTOCOMPLETE	75.7%	4.64	7.46
$K_{\max}$=1	80.4%	10.26	35.28
`max_resamples`=100	77.3%	4.81	9.36
`max_resamples`=50	75.5%	4.74	7.80

Table 2: Component ablation on JSB medium_test (n=511, LLaDA-8B-Instruct, B200, T=128, seed=0). Latencies in seconds. Top block uses per-position AR autocomplete (the Table 1 setting); bottom block uses the greedy block-completion variant for comparison.

The paired outcomes in Figure 2 show where the validity gains and remaining failures come from. Among the 511 common JSB medium_test instances, Dgrammar and LAVE agree on 405 cases (358 both pass, 47 both fail). Dgrammar wins 74 cases that LAVE misses, while LAVE wins 32 cases that Dgrammar misses. Thus the improvement is not because one method dominates every schema, but because Dgrammar's wins are concentrated in LAVE's expensive failure mode: schemas where stochastic lookahead repeatedly rejects constrained continuations and often reaches the 120 s timeout. On the 65 LAVE timeout instances, Dgrammar never times out and 46 become schema-valid, at a median of 5.2 s, a 23× speedup over the LAVE timeout.

Method	Split	Fails	Empty	Parse	Schema
LAVE	medium	110	33	77	0
DGR.	medium	90	0	90	0
LAVE	hard	113	40	73	0
DGR.	hard	120	0	120	0

Table 3: Dream-7B failure decomposition on JSB medium (n=506) and hard (n=258). Both decoders have zero schema errors; all failures occur before validation. Dgrammar eliminates empty timeout outputs, but its remaining failures are long parse errors (mean 577/761 chars), beyond the 256-token budget. The Dream hard regime thus mostly tests generation length rather than constraint quality.

The remaining 32 LAVE-only successes expose the main boundary of our method (Table 3 gives the same decomposition for Dream). On these instances, LAVE's mean runtime is 21.8 s (max 87.3 s, well below the timeout), so they are genuine constraint-quality losses rather than easy cases LAVE happened to find first. Re-validating Dgrammar's failed outputs shows all of them fail at JSON parsing rather than schema validation: the median failed output is 578 characters long, exceeding the 256-token budget, so the dominant residual failure mode is tail closure on long-output schemas (Table 3) rather than field selection. Lowering `max_resamples` barely moves validity under the AR autocomplete fallback (84.5%→84.1% at budget 100, →83.6% at budget 50); the cost shifts to latency instead, since more instances run out of resample budget and trigger the per-token AR completion (mean 5.24→9.55→11.55 s). Conversely, increasing the budget can recover validity on harder backbones, as in Dream medium_test, but at a measurable latency cost. Dgrammar should therefore be read as a better validity–latency Pareto point, not as a universal validity winner over lookahead verification on every split.

Case study. Instance `o83272` is representative of the timeout wins (App. C). Its schema contains enum and format-constrained fields such as `resource`, integer counts, and date strings. LAVE must verify a frontier token by drawing random completions; for an enum field this requires a free-form completion to hit a specific token such as `"CPU"`, so every lookahead batch fails and the instance reaches the 120 s timeout. Dgrammar instead masks the frontier vocabulary to enum-valid tokens, completes the instance in 3.39 s with zero

resamples, and produces schema-valid output.

Empirical intervention rate. Figure 3 compares per-instance intervention counts on JSB medium_test (LLaDA-8B). LAVE's lookahead retries have a long tail (median 16, mean 261, max 3334), with each retry costing $K{=}5$ extra forward passes. Dgrammar's selective remasks are much more compact in the body (median 1; 43.4% of instances need zero resamples and complete the diffusion loop without any frontier repair) and bounded by `max_resamples` in the tail; each remask is an argmax on existing logits, with no additional forward pass. The intervention machinery therefore engages rarely on most instances and, when it does, perturbs only the violating frontier token rather than the whole frontier scoring rule.

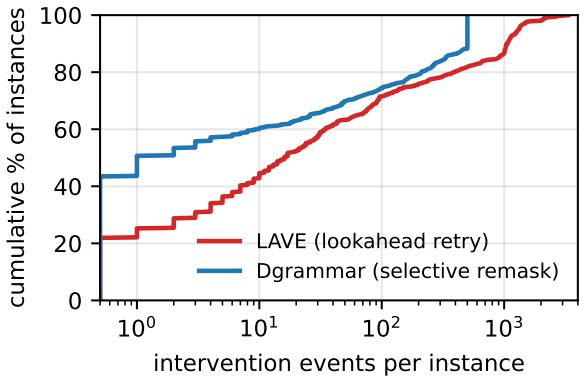

Figure 3: Empirical CDF of intervention events per instance on JSB medium_test (LLaDA-8B-Instruct, $n{=}511$). LAVE retries are lookahead-verification draws ($K{=}5$ forward passes each); Dgrammar resamples are selective remasks (no extra forward pass). Dgrammar's median is 1 vs. LAVE's 16; Dgrammar bounds its tail at `max_resamples`=500.

5.3 Transfer and Runtime Overhead

Difficulty spectrum. Across all three JSB splits and both backbones, Dgrammar matches or exceeds LAVE on validity while always running with zero timeouts. On LLaDA, the validity advantage is +11.8/+8.2/+1.1 pp on easy/medium/hard; on Dream the pattern is similar (+10.5/+3.9/+0.4 pp), with the gap narrowing on hard schemas where stochastic lookahead is more likely to find a valid completion. LAVE's timeout rate stays roughly stable across difficulty (14.9%/12.7%/20.8% on LLaDA), indicating its long-tail failure mode is structural (e.g., enum/format constraints) rather than tied to JSB's difficulty labels.

Dataset	Method	syn@1	fun@1
LLaDA-8B-Instruct			
JSON	Vanilla	85.7	46.7
JSON	LAVE	98.5	52.9
JSON	DGR.	**99.3**	52.2
SMILES	Vanilla	65.3	2.4
SMILES	LAVE‡	98.7	0.0
SMILES	DGR.	**100.0**	**3.0**
Dream-7B			
JSON	Vanilla	86.8	49.6
JSON	LAVE	95.9	45.8
JSON	DGR.	94.8	**45.2**†
SMILES	Vanilla	79.6	**3.6**
SMILES	LAVE‡	99.4	0.0
SMILES	DGR.	**99.4**	2.4

Table 4: Cross-benchmark validity. JSON-Bench ($n{=}272$) and SMILES ($n{=}167$); B200, same hyperparameters as Table 1. SMILES syn@1 follows Zhang et al. (2026)'s CFG-acceptance metric; fun@1 is RDKit canonical equivalence. LAVE uses its published default (`max_retry_num_total`=1000); the tuned $N{=}10, \tau{=}10$ in Zhang et al. (2026) gives higher fun@1 (55.0/54.4 on LLaDA/Dream JSON). †LAVE Dream-7B JSON timeouts = 8; Dgrammar = 0. ‡LAVE SMILES syn@1 from Zhang et al. (2026) Table 1; that paper excludes SMILES from fun@1 (all methods round to 0%).

Functional correctness. On the JSON-Bench split (`json-mode-eval-extended`, $n{=}272$) used by LAVE (Zhang et al., 2026), Dgrammar's functional@1 (exact match against ground truth) is statistically indistinguishable from LAVE's: 52.2 vs. 52.9 on LLaDA-8B and 45.2 vs. 45.8 on Dream-7B (both within 0.7 pp; case-insensitive: 61.1 vs. 61.0 and similar pattern on Dream). Both also clear the unconstrained ceiling: VANILLA's fun@1 is 46.7/49.6 on LLaDA/Dream, so Dgrammar adds +5.5/−4.4 pp[1] on top of the model's own success rate. Because LAVE applies grammar enforcement only as a stochastic verifier on top of the same base model, this match is the right reference point: it shows Dgrammar's deterministic frontier masking does *not* cost semantic accuracy beyond the model's own ceiling. Latency, in contrast, drops 17–75% on LLaDA and 3×/10× (mean/p95) on Dream, with zero timeouts vs. LAVE's eight on Dream.

[1]The Dream regression is within the 0.7 pp gap to LAVE noted above and tracks LAVE's own behavior on this split, suggesting it reflects sampling variance under the temperature=0.2 setting rather than a constraint cost.

Operation	Mean
grammar check (per call)	0.15 ms
mask compute (per call)	13.23 ms
mask wait after async overlap	0.05 ms
mask time saved (per instance)	1034 ms
autocomplete fallback fires	26.2%

Table 5: Per-operation overhead for Dgrammar on JSB medium_test.

Cross-grammar transfer (SMILES). To test that Dgrammar's algorithm is not specific to JSON-Schema, we plug in a CFG-mode checker (rustformlang-backed CFG/DFA intersection-emptiness) and run on `eth-sri/smiles-eval` (n=167, Table 4). Dgrammar reaches 100.0% syn@1 on LLaDA-8B (+34.7 pp over VANILLA, also above LAVE's 98.7%) and 99.4% on Dream-7B (matching LAVE), confirming the algorithm transfers to non-JSON grammars. Functional@1 stays at 2–4 % across all methods because the SMILES grammar accepts any single atom as a valid `Line`, and exact-match against a stereoisomeric reference is dominated by the model's chemistry knowledge rather than constraint enforcement; Zhang et al. (2026) likewise reports 0% functional on this split. The full Dgrammar pipeline transfers across backbones via only a per-model adapter (forward signature, mask/EOS token IDs, tokenizer); see Table 4.

Table 5 reports per-operation timing for Dgrammar on JSB medium_test. The grammar check averages 0.15 ms per call, three to four orders of magnitude below a forward pass. Mask construction averages 13.2 ms but is overlapped with the forward pass on every step where the frontier is masked, saving on average about 1.0 s per instance and reducing the observed mask wait to 0.05 ms. The autocomplete fallback fires on 26.2% of instances; its cost is bounded because it only completes the suffix that the diffusion loop failed to close. On the AR run (Table 1), the AR completion produces a median of 21 tokens per fired instance (mean 72.6, max 245) and accounts for only 8.8% of all generated tokens, so most output is still produced by the multi-token diffusion loop.

6 Conclusion

We argued that per-token constraint overhead is a useful axis for comparing grammar-constrained decoders on DLMs, and built Dgrammar, a decoder that preserves multi-token unmasking while enforcing token-level grammar via incremental prefix checking and selective remasking. On JSONSchemaBench, Dgrammar both improves schema-valid rate over the strongest existing DLM baseline and reduces tail latency by an order of magnitude. The overhead breakdown suggests that, once mask construction is overlapped, the dominant systems risk is not grammar checking itself but extra model forwards caused by retries, fallbacks, or token-by-token commitment. We will release the full implementation, including the asynchronous mask overlap, AIMD $K_{\max}$ schedule, and matcher integration, upon publication.

Future directions. Three extensions follow naturally: a stronger autocomplete fallback (short beam search or a learned tail-completion head) to narrow the long-output validity gap, integration with parallel-decoding accelerators (Wu et al., 2025; Chen et al., 2026, 2025) whose adaptive-K schedules our mask-based repair should be compatible with, and scaling Dgrammar to larger CFGs (Python, C++) via vectorized DFA or sparse-transition compilation.

Limitations

Coverage and grammar support. We evaluate on two DLMs and three benchmark families. Table 4 shows transfer from JSON-Schema to SMILES via a CFG-mode checker, but we have not yet tested larger code grammars such as C++ or Python, where grammar size may dominate latency. Dgrammar also inherits llguidance's schema coverage: 75 recursive `$ref+oneOf` JSB schemas cannot be compiled and are skipped by all methods. We include ETH/IG-CD (Mündler et al., 2025) in Table 1 but it is not instance-matched: its regex backend rejects 22–35% of each split (e.g. `\/` URI escapes and null-bounded length quantifiers), and even on the schemas it does compile it trails Dgrammar at every difficulty on both backbones while hitting 50/245 (LLaDA) and 35/245 (Dream) timeouts on hard. We omit DINGO (Suresh et al., 2025) because no runnable implementation was released at submission time. Finally, all runs use a matched `seed=0`; multi-seed sweeps, LAVE hyperparameter sweeps, and grammar-size scaling are left for future work.

Algorithm and semantic ceiling. The autocomplete fallback is the main correctness frontier; better fallbacks such as short beam search or learned completion heads may improve tail closure. Grammar constraints guarantee *structural* validity, not semantic quality: valid outputs can still be empty arrays, empty objects, short strings, or otherwise uninformative, contributing to the gap between schema@1 and functional@1 (Table 4). We also do not claim that Dgrammar samples from the constrained joint distribution; frontier-only masking is a limited bias, not a zero-bias procedure. In deployment, Dgrammar should be treated as a structural enforcement layer, with downstream semantic and safety checks applied as usual.

References

Zigeng Chen, Gongfan Fang, Xinyin Ma, Ruonan Yu, and Xinchao Wang. 2025. dParallel: Learnable parallel decoding for dLLMs. *arXiv preprint arXiv:2509.26488*.

Zigeng Chen, Gongfan Fang, Xinyin Ma, Ruonan Yu, and Xinchao Wang. 2026. DMax: Aggressive parallel decoding for dLLMs. *arXiv preprint arXiv:2604.08302*.

Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. 2024. XGrammar: Flexible and efficient structured generation engine for large language models. *arXiv preprint arXiv:2411.15100*.

Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. 2025. JSONSchemaBench: A rigorous benchmark of structured outputs for language models. *arXiv preprint arXiv:2501.10868*.

Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Miao Li, Hanyang Jiang, Sikai Cheng, Hengyu Fu, Yuhang Cai, Baihe Huang, Tinghan Ye, Xuanzhou Chen, and Pascal Van Hentenryck. 2026. Plan, verify and fill: A structured parallel decoding approach for diffusion language models. *arXiv preprint arXiv:2601.12247*.

Microsoft. 2024. llguidance: Super-fast constrained decoding. https://github.com/guidance-ai/llguidance.

Niels Mündler, Jasper Dekoninck, and Martin Vechev. 2025. Constrained decoding of diffusion llms with context-free grammars. *arXiv preprint arXiv:2508.10111*.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. 2025. Large language diffusion models. *arXiv preprint arXiv:2502.09992*.

Kanghee Park, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D'Antoni. 2024. Grammar-aligned decoding. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Kanghee Park, Timothy Zhou, and Loris D'Antoni. 2025. Flexible and efficient grammar-constrained decoding. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*.

Avinash Reddy, Thayne T. Walker, James S. Ide, and Amrit Singh Bedi. 2026. Draft-conditioned constrained decoding for structured generation in LLMs. *arXiv preprint arXiv:2603.03305*.

Tarun Suresh, Debangshu Banerjee, Shubham Ugare, Sasa Misailovic, and Gagandeep Singh. 2025. DINGO: Constrained inference for diffusion LLMs. *arXiv preprint arXiv:2505.23061*.

Brandon T. Willard and Rémi Louf. 2023. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*.

Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. 2025. Fast-dLLM: Training-free acceleration of diffusion LLM by enabling kv cache and parallel decoding. *arXiv preprint arXiv:2505.22618*.

Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. 2025. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*.

Yitong Zhang, Yongmin Li, Yuetong Liu, Jia Li, Xiaoran Jia, Zherui Li, and Ge Li. 2026. Lookahead-then-verify: Reliable constrained decoding for diffusion LLMs under context-free grammars. *arXiv preprint arXiv:2602.00612*.

A Hyperparameters

Table 6 lists shared hyperparameters across all methods on JSB medium_test.

Hyperparameter	Value
Diffusion steps T	128
Random seed	0
Block length	32
Generation length	256
Remasking strategy	low-confidence
Temperature	0.2
Max resamples (LAVE)	1000
Max resamples (Dgrammar)	500
$K_{\max}$ (multi-token unmask)	8
Per-instance timeout	120 s

Table 6: Shared hyperparameters for benchmark runs.

B Dataset and Instance Filtering

JSB medium_test contains 586 schemas; 75 are excluded from all comparisons because llguidance rejects them at compile time (recursive `$ref+oneOf` structures). LAVE returns immediately with `valid=false` (`"Prompt does not conform to grammar"`); Dgrammar silently skips them via the same llguidance guard. Since all methods behave identically on these instances, including them would inflate failure counts without providing meaningful comparison. All headline numbers are reported on the matched n=511 subset.

C Case Study: Instance o83272

We examine instance `o83272`, a cluster resource allocation schema where LAVE times out and Dgrammar succeeds in a single pass with zero resamples. The schema requires fields with type-constrained values: `resource` (enum string), integer fields (`nodes`, `processors`, `ppn`, `percent_allocated`), and date strings.

Why LAVE fails (timeout, 120 s, no output). LAVE commits a token only if all K randomly sampled completions yield a grammar-valid continuation. For enum or typed fields, this requires a random natural-language completion to match a specific technical token (e.g., `"CPU"` for `resource`). LLaDA-8B assigns negligible probability mass to such targeted strings as free continuations, so every lookahead batch fails. LAVE cannot commit even the first constrained token and hits the 120 s wall without producing output.

Dgrammar (valid, 0 resamples, 3.39 s). Frontier masking restricts the vocabulary at each position to the set of tokens permitted by the matcher. For `resource`, only the enum-valid tokens are unmasked; for integer and date fields, only digits and the appropriate format tokens are exposed. Because the matcher guides every token choice, no violation occurs and the selective-remasking path is never triggered. The output is complete and schema-valid:

```
{"resource": "CPU", "nodes": 4, "processors": 16,
 "ppn": 4, "start_date": "2023-10-01",
 "end_date": "2023-10-31", "percent_allocated": 75,
 "comments": "..."}
```

Summary. This is the general failure mode of stochastic lookahead on enum and format-constrained fields: the probability of a random completion matching a specific technical string is effectively zero, so LAVE never commits, whereas frontier masking sidesteps this entirely (the aggregate timeout-recovery numbers are in §5).

D EOS-Validity vs. Schema-Validity

A subtle but important distinction motivates our choice of metric. The runner-internal `valid` flag tests only whether an EOS token is present and no `[MASK]` remains before it; this is necessary for grammar-valid output but not sufficient when schemas carry *format* constraints (e.g., `format: "uuid"`, `format: "uri"`) that llguidance compiles into the token automaton.

For example, on a schema requiring `target: {type: string, format: "uuid"}`, Dgrammar can emit a token sequence whose individual tokens each satisfy the automaton's prefix condition, but whose concatenation, e.g., `"12000e1TestCaseCanceledEvent"`, does not match the full RFC 4122 pattern. The EOS check passes; `jsonschema.validate` on the parsed output fails. We therefore report only *schema@1*, the result of `jsonschema.validate`, in all main tables; the same definition is used for LAVE. This is the same metric used by JSONSchemaBench (Geng et al., 2025).

E Engineering Details

AIMD $K_{\max}$ schedule. With $K_{\max}$=8, T=128, and block length 32, the AIMD rule of §4 converges to K=2–3 on average ($\bar{K} \approx 1.96$ in our runs).

Async mask overlap. `compute_mask` runs on a CPU worker launched right after the previous step's logits commit, so the next GPU forward starts without waiting; observed mask wait then drops from 13.2 ms (compute-bound) to 0.05 ms (sync-bound).

Per-backbone adapter. Model-specific bits sit behind three hooks: `forward_fn` (on Dream, a one-position causal shift aligning logits with the masking convention), the `(mask_id, eos_id)` pair, and the tokenizer; no other code paths differ across LLaDA-8B and Dream-7B.